System control implementation specification (pe_sys_ctrl) v1.1

Nov 10, 2026 · @Aries X

0. Status and figures

pe_sys_ctrl is 30 flops and about 2.7k μm² (estimate), never holds the bus, and implements every decision taken with the project owner on 10 Nov 2026. This spec adds exact equations, cycle timing, area and the verification plan; the equations pass a Python gate-equation model (scratch `sc_gates.py`, not the reference model: system control is not in pe.py or mc.py). No synthesis; areas are estimates.

v1.1 (10 Nov 2026): the bus target port uses the same 7-signal interface as the host link (req, dir, addr, wdata, wready, rready, rdata), with the unused parts tied off inside the block (owner). Read data follows the protocol's timing (valid the clock after acceptance); this adds 4 flops (D-SYS-10).

There is no separate micro-architecture chapter for this block; the decision log is in this document. Not final yet: how the interconnect drives the target port (§2.1, assumption AS-1, waiting for the bus chapter) and four decisions taken while writing (§13, D-SYS-7 to D-SYS-10), which need the owner's confirmation.

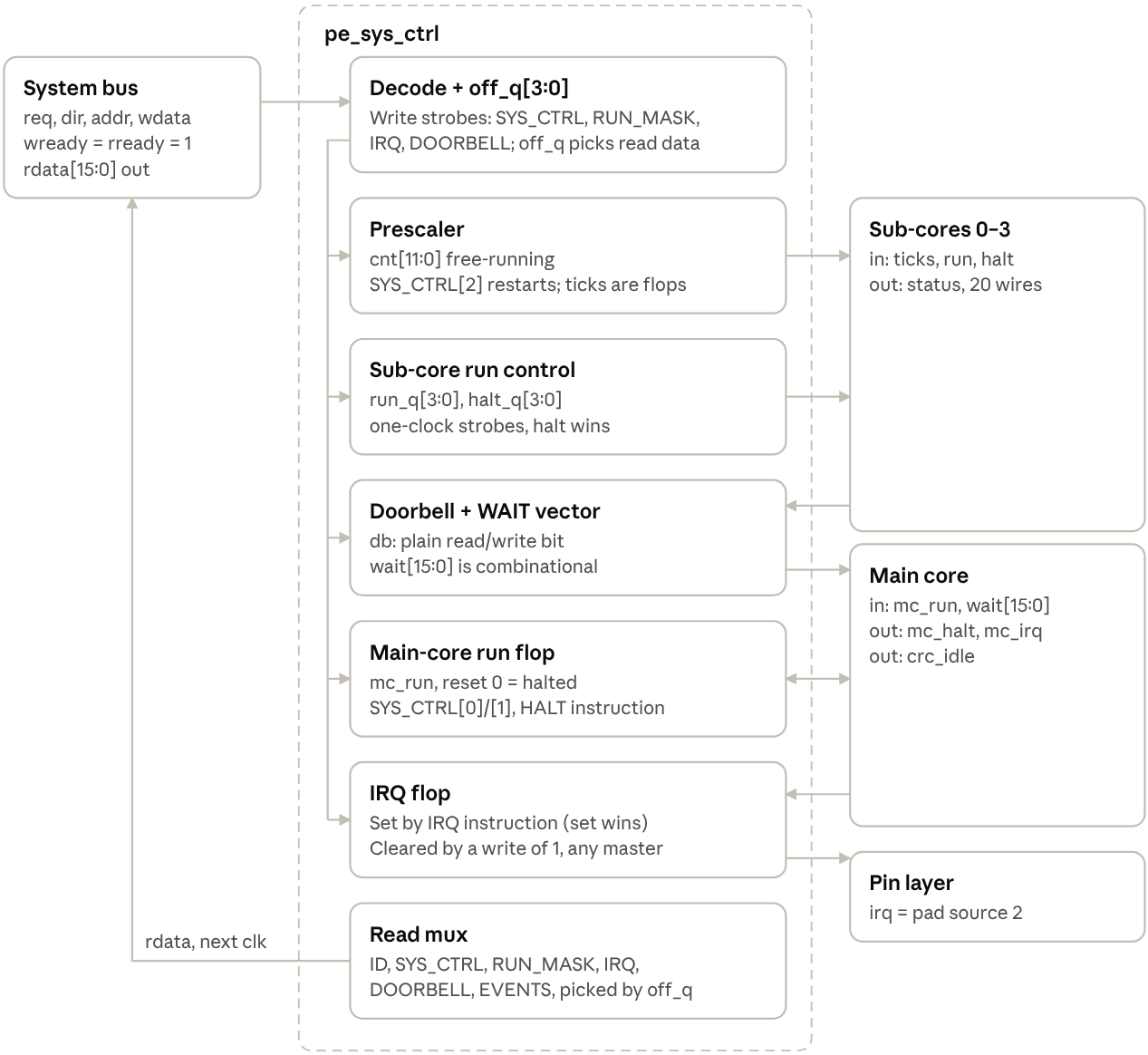
Figure	Where	Source
F1 Micro-architecture concept	§1	claude/sys-ctrl-uarch.drawio
T1 Prescaler restart	§7	Appendix A, wd1_prescaler
T2 RUN_MASK same-clock start	§7	Appendix A, wd2_run_mask
T3 Main-core run and halt	§7	Appendix A, wd3_mc_run
T4 IRQ set, clear and set-wins	§7	Appendix A, wd4_irq
T5 Doorbell to WAIT wake-up	§7	Appendix A, wd5_doorbell
T6 Bus write and read	§7	Appendix A, wd6_bus

1. Approved changes, scope and registers

System control owns six registers at 0x1C00–0x1C05, the shared prescaler, the run control of the main core and the sub-cores, the IRQ and doorbell flops and the WAIT vector (figure F1). Two architecture changes were approved for it.

Bus writes fan out to five functions; all outputs but WAIT leave from flops

F1 pe_sys_ctrl micro-architecture · 30 flops · registers 0x1C00–0x1C05 · never holds the bus



F1 · pe_sys_ctrl micro-architecture

The decode block's write strobes run down the spine on the left to the five register functions; sub-core strobes, ticks, IRQ and mc_run leave from flops, and only the WAIT vector is combinational. Source: claude/sys-ctrl-uarch.drawio.

1.1 Architecture changes (owner, 10 Nov 2026)

ID	Change	Replaces	Reason
A-CHG-3	Any bus master clears IRQ by writing 1 to bit 0	Arch §6.4 and requirements §6: "host writes 1 to clear"	The main core can withdraw a stale IRQ; no master ID is needed in sys_ctrl
A-CHG-4	SYS_CTRL and RUN_MASK are readable and return live state (§1.3)	Requirements §6: both marked W	Software can see run state in one read

architecture-1.0.md §6.4, uarch-requirements-1.0.md §6 and claude/uarch-memory-map.md §5 must be amended to match.

1.2 Points the architecture left unspecified, now decided

ID	Decision
S-1	EVENTS[14] stays as written (level: any sub-core halted or with an error flag). Software rule in §11
S-2	A write that sets both the run and the halt bit of one target: halt wins. Applies to RUN_MASK and SYS_CTRL
S-3	Run for the partner of a 32/0 split is ignored. Implemented in sc_pair, not here (§2.2)
S-4a	DOORBELL is a plain read/write bit: bit 0 takes the written value, from any master
S-4c	IRQ set and clear in the same clock: set wins
M-2	ID = 0x5010 (O-05)

1.3 Register behaviour (0x1C00 + offset)

Off	Name	Write	Read	Reset
0x0	ID	ignored	0x5010	—
0x1	SYS_CTRL	[0] 1 = run main core, [1] 1 = halt main core, [2] 1 = restart prescaler. 0 bits do nothing	[0] main core running, [1] main core halted, [15:2] 0	reads 0x0002
0x2	RUN_MASK	[3:0] 1 = run SCn, [7:4] 1 = halt SCn. 0 bits do nothing	[3:0] SCn running, [7:4] SCn halted, [15:8] 0	reads 0x00F0
0x3	IRQ	[0] 1 = clear	[0] IRQ	0
0x4	DOORBELL	[0] stored	[0] doorbell	0
0x5	EVENTS	ignored	WAIT vector (§5.5)	—
0x6–0xF	unused	ignored	0	—

Unused write bits are ignored. No read has a side effect. The block never holds the bus (D-6).

1.4 Not in this block

- CRC unit (0x1C10–0x1C15): moved under `main_core` (module and chapter). The interconnect decodes its range directly. This changes the hierarchy in requirements §2, not behaviour.
- SIG lines: wiring between sub-cores (sideband), not routed through this block.
- The sub-cores' halted flops and the 32/0 partner gating: in subcore / sc_pair.

2. Interface

The bus side is the same 7-signal port as the host link's master port; everything else is point-to-point wiring to the main core, the sub-cores and the pin layer.

```
module pe_sys_ctrl (
  input  wire        clk,
  input  wire        rst_n,          // synchronised reset (G-02)
  // bus target, 0x1C00-0x1C0F: same 7 signals as the host link's master port
  input  wire        req_i,          // access to this block requested this
clock (AS-1)
  input  wire        dir_i,          // 1 = write, 0 = read
  input  wire [15:0] addr_i,         // only [3:0] used; [15:4] tied off
inside
  input  wire [15:0] wdata_i,
  output wire        wready_o,       // tied 1: never waits
  output wire        rready_o,       // tied 1: never waits
  output wire [15:0] rdata_o,        // valid the clock after the read is
accepted
  // main core
  output wire        mc_run_o,       // 1 = main core executes this clock
  input  wire        mc_halt_i,      // HALT instruction executes this clock
  input  wire        mc_irq_i,       // IRQ instruction executes this clock
  output wire [15:0] wait_o,         // WAIT vector, bit 15 = 0
  input  wire        crc_idle_i,
  // sub-cores
  output wire [3:0]  sc_run_o,       // one-clock strobes, from flops
  output wire [3:0]  sc_halt_o,      // one-clock strobes, from flops
  output wire [2:0]  tick_o,         // [0] /16, [1] /256, [2] /4096, from
flops
  input  wire [3:0]  sc_fa_space_i,  // FIFO A count < depth
  input  wire [3:0]  sc_fb_ne_i,     // FIFO B not empty
  input  wire [3:0]  sc_ntf_i,       // any notify flag set
  input  wire [3:0]  sc_halted_i,    // halted flop
  input  wire [3:0]  sc_err_i,       // any error flag set
  // pin layer
  output wire        irq_o           // pad source 2, from a flop
);
```

2.1 Bus target port

The port has the same seven signals, names and meanings as the host link's master port (impl-spec-host-link §2, D-HL-7), seen from the target side:

Signal	Dir	Use in this block
req_i	in	Used. An access to this block is presented this clock
dir_i	in	Used. 1 = write, 0 = read (owner, 10 Nov 2026)
addr_i[15:0]	in	Bits [3:0] used. Bits [15:4] are tied off (left unconnected inside; synthesis removes them)
wdata_i[15:0]	in	Bits [7:0] used (SYS_CTRL [2:0], RUN_MASK [7:0], IRQ [0], DOORBELL [0]). Bits [15:8] tied off
wready_o	out	Tied to 1. The block never holds the bus (D-6)
rready_o	out	Tied to 1. The block never holds the bus (D-6)
rdata_o[15:0]	out	Valid in the clock after the read is accepted, as the host link expects (`rpend` captures it then). Bit 15 is constant 0

Accept rule: with both ready outputs tied to 1, an access is accepted in every clock where `req_i` = 1. A write (`dir_i` = 1) takes effect at the edge ending that clock (BUS-02). A read (`dir_i` = 0) has no side effect; its data is on `rdata_o` in the next clock (figure T6).

Assumption AS-1 (for the bus chapter): the interconnect decodes addr[15:4] = 0x1C0 and passes `req` to this block only for such accesses (one-hot select); addr, dir and wdata are broadcast. It forwards the selected target's ready, and in the clock after a read it selects this block's `rdata_o`. If the bus chapter gives every target the raw `req` instead, this block adds the decode of addr[15:4] (one 12-bit compare, about 0.1k μm²); nothing else changes.

No master ID (`bid`) is needed: after S-4a and A-CHG-3 nothing in this block depends on which master wrote.

2.2 Requirements on the other blocks

Block	Requirement
subcore	Owns its halted flop. `sc_run_o[n]` = 1 in a clock: running from the next edge. `sc_halt_o[n]` = 1: halted from the next edge. sys_ctrl never raises both for one n. Priority against CTRL writes, the HALT micro-op and the breakpoint in the same clock is defined in the sub-core chapter
subcore	Sends `halted` and `any error flag` as two wires. SC-STS-05 lists one combined "halted OR error" wire; A-CHG-4 needs the halted flag alone (4 extra wires)
sc_pair	S-3: while the master's SPLIT = 32/0, the partner ignores `sc_run_o`, CTRL.RUN and CTRL.STEP
main_core	Executes only in clocks with `mc_run_o` = 1. Raises `mc_halt_i` / `mc_irq_i` for exactly the clock in which a HALT / IRQ instruction executes (not in stall clocks, not while `mc_run_o` = 0). What happens to a multi-clock instruction when `mc_run_o` falls in its middle is for the main-core chapter
pin_layer	`irq_o` is input 2 of every pad select, drive enable 1 (PL-03)

3. Storage

The block holds 30 flops, all sg13cmos5l dfrbpq (CLK = clk, RESET_B = rst_n, reset 0), and no latches.

Flop	Bits	Purpose
cnt[11:0]	12	Prescaler counter, free-running
tick[2:0]	3	Registered ÷16, ÷256, ÷4096 ticks (D-1)
run_q[3:0]	4	Sub-core run strobes (D-3)

Flop	Bits	Purpose
halt_q[3:0]	4	Sub-core halt strobes (D-3)
mc_run	1	Main core running (D-5). Reset 0 = halted
irq	1	Host IRQ
db	1	Doorbell
off_q[3:0]	4	addr[3:0] of the previous clock: selects the read data in the clock after a read (D-SYS-10)

No clock gating: every enable is logic in front of D. `mc_run` is stored with run polarity so that reset 0 means halted (SYS-01).

4. Decode

```
wr     = req_i & dir_i                          (wready_o = 1, so every
requested write is accepted)
a      = addr_i[3:0]
wr_sys = wr & (a == 1)    wr_rm = wr & (a == 2)    wr_irq = wr & (a == 3)
wr_db = wr & (a == 4)
off_q_d = a                                     (every clock, no enable)
wready_o = 1      rready_o = 1
```

`off_q` loads every clock without an enable. In the clock after an accepted read it holds that read's offset, which is the only clock in which the interconnect uses `rdata_o`. In every other clock `rdata_o` is don't-care.

5. Logic

Every flop group follows one short equation; there is no state machine.

5.1 Prescaler (SYS-03, D-1, D-2)

```
restart  = wr_sys & wdata_i[2]
c4       = &cnt[3:0]
c8       = c4 & &cnt[7:4]
c12      = c8 & &cnt[11:8]
cnt_d    = restart ? 12'd0 : cnt + 1          (c4 and c8 are the
incrementer's group carries)
tick_d   = { restart | c12, restart | c8, restart | c4 }
tick_o   = tick
```

`tick[0]` is 1 exactly in the clocks where cnt[3:0] = 0, `tick[1]` where cnt[7:0] = 0, `tick[2]` where cnt[11:0] = 0. That is the SYS-03 definition, taken from flops instead of a decode of cnt. One exception: in the first clock after reset cnt = 0 and tick = 0. It has no effect, because no timer runs then.

The ÷1 tap is not sent; a timer with PSC = 0 uses constant 1 (SC-TMR-01).

5.2 Sub-core run control (SYS-02, S-2, D-3)

```
run_q_d[n]  = wr_rm & wdata_i[n] & !wdata_i[4+n]        n = 0..3   (halt
wins)
halt_q_d[n] = wr_rm & wdata_i[4+n]
sc_run_o    = run_q        sc_halt_o = halt_q
```

Each strobe is high for one clock, the clock after the write.

5.3 Main-core run flop (MC-05, S-2, D-5)

```
mc_set   = wr_sys & wdata_i[0]
mc_clr   = (wr_sys & wdata_i[1]) | mc_halt_i
mc_run_d = mc_clr ? 0 : mc_set ? 1 : mc_run
mc_run_o = mc_run
```

SYS_CTRL acts on `mc_run` directly, with no strobe flops in between (D-SYS-7). A HALT instruction in the same clock as a SYS_CTRL run write leaves the core halted (D-SYS-8).

5.4 IRQ and doorbell (SYS-04, A-CHG-3, S-4a, S-4c)

```
irq_d = mc_irq_i | (irq & !(wr_irq & wdata_i[0]))        (set wins)
db_d  = wr_db ? wdata_i[0] : db
irq_o = irq
```

5.5 WAIT vector (SYS-05, D-4)

```
wait_o[3:0]   = sc_fa_space_i
wait_o[7:4]   = sc_fb_ne_i
wait_o[11:8]  = sc_ntf_i
wait_o[12]    = db
wait_o[13]    = crc_idle_i
wait_o[14]    = |(sc_halted_i | sc_err_i)
wait_o[15]    = 0
```

Combinational, with no flop of its own, so WAIT wakes in the same clock as `mc.py`'s live `events()`.

5.6 Read mux

```
rdata_o = off_q == 0 ? 16'h5010
        : off_q == 1 ? {14'b0, !mc_run, mc_run}
        : off_q == 2 ? {8'b0, sc_halted_i, ~sc_halted_i}
        : off_q == 3 ? {15'b0, irq}
        : off_q == 4 ? {15'b0, db}
        : off_q == 5 ? wait_o
        : 16'h0000
```

Built as AND-OR per bit. Only bit 0 has five sources; bits 8–11, 13 and 15 have one or none.

The value returned is the state at the start of the clock after acceptance, just after the edge that accepted the read. No other bus access can be accepted at that edge (one access per clock, BUS-01), so a read sees every write accepted before it and none after it. All sources are levels without read side effects, so reading them one clock later than a combinational read would is not visible to software.

6. State

The run state of the system lives in two places: `mc_run` here for the main core, and each sub-core's own halted flop.

Target	Flop	Set running by	Set halted by	Both in one clock
Main core	`mc_run` (here)	SYS_CTRL[0]	SYS_CTRL[1], HALT instruction, reset	halted
Sub-core n	halted flop (in the sub-core)	RUN_MASK[n], CTRL.RUN, CTRL.STEP (one clock)	RUN_MASK[4+n], CTRL.HALT, HALT micro-op, breakpoint, reset	RUN_MASK: halted (S-2). CTRL: RUN wins (SC-WIN-02). Across sources: sub-core chapter

RUN_MASK and CTRL resolve a run + halt conflict in opposite ways. This is deliberate (owner decision S-2 against the frozen SC-WIN-02); software should never write both bits for one target.

7. Cycle timing

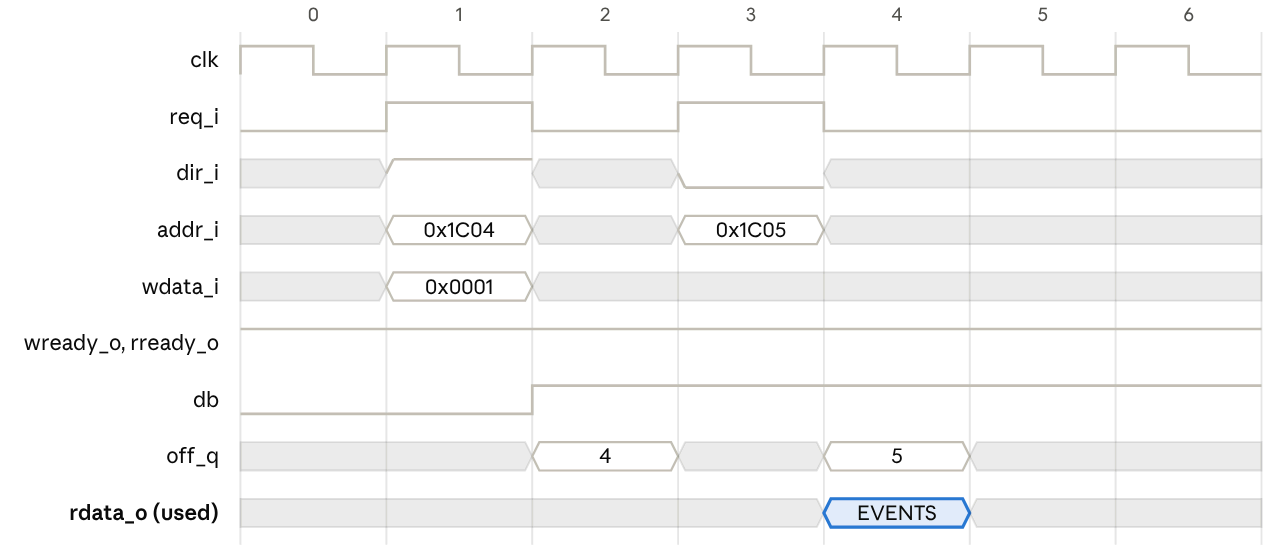
Every register write takes effect at the edge ending its accept clock W; sub-cores start in W + 2, the main core in W + 1, and read data arrives in the clock after the read.

7.1 Bus access

Clock	Write	Read
A (req_i = 1)	wready_o = 1: accepted; takes effect at the edge ending A	rready_o = 1: accepted; off_q ← addr[3:0] at the edge ending A
A + 1	new register value visible	rdata_o valid (host link: `rpend` loads it)

A write lands at the end of its clock; read data arrives one clock later

T6 Bus write and read · one cell = one clk



Write accepted in clock 1, db = 1 from clock 2. Read accepted in clock 3, off_q = 5 and rdata valid in clock 4

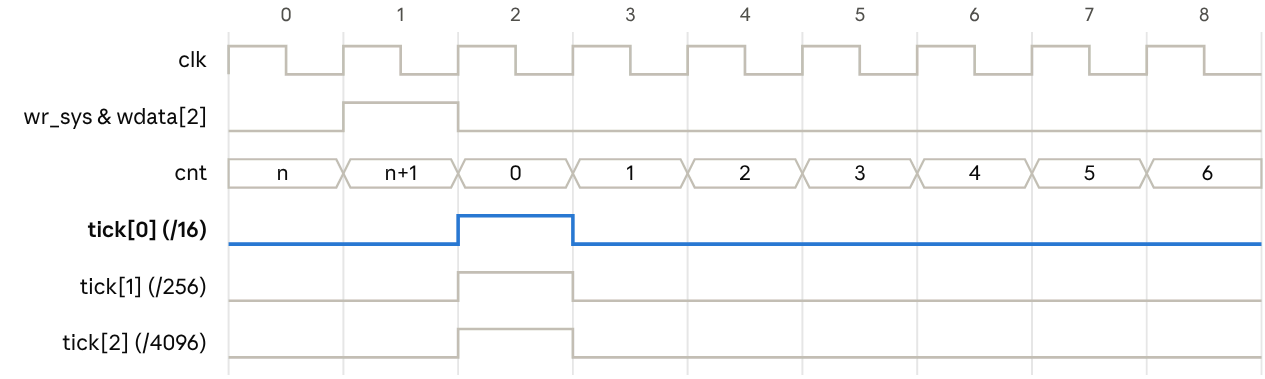
T6 · bus write then read, no wait

7.2 Prescaler restart, SYS_CTRL[2]

Clock	Event
W	`restart` = 1
W + 1	cnt = 0, all three ticks = 1
W + 1 + 16k	÷16 tick (÷256 and ÷4096 likewise)

A prescaler restart pulses all three ticks in the next clock

T1 Prescaler restart · one cell = one clk



All three ticks pulse in W + 1 (cnt = 0); the next /16 tick is in W + 17, /256 in W + 257, /4096 in W + 4097

T1 · prescaler restart by SYS_CTRL[2]

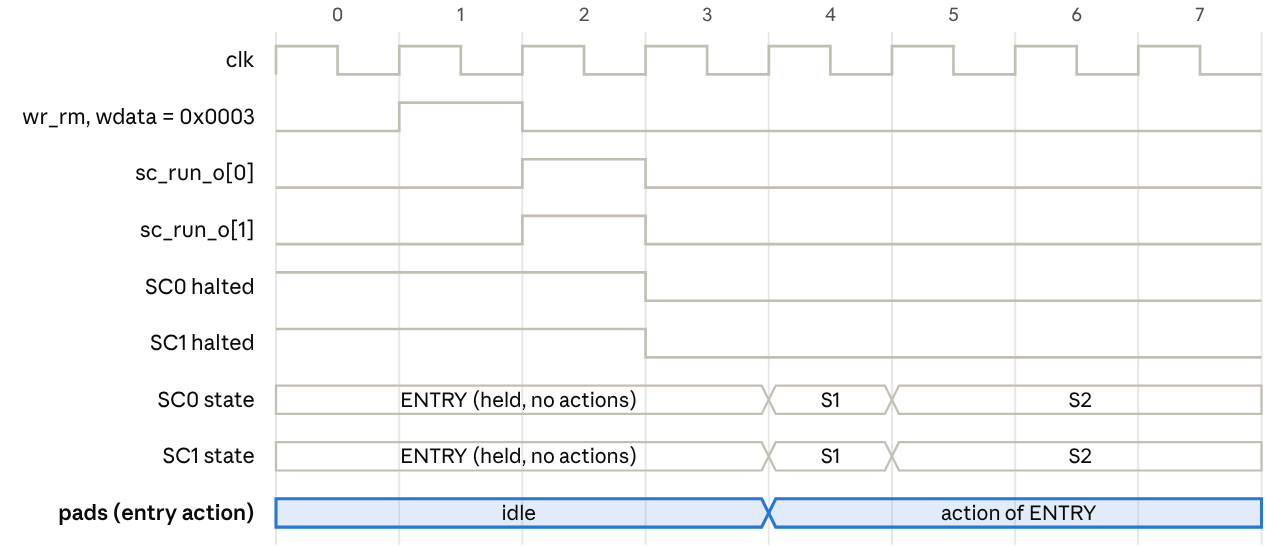
7.3 Sub-core start, RUN_MASK

Clock	Event
W	`wr_rm` = 1
W + 1	`sc_run_o[n]` = 1 for every selected n
W + 2	Every selected sub-core is running and executes its first state
W + 3	That state's pin action is on the pad (G-04)

Halt has the same timing: the last executed clock is W + 1. A RUN_MASK read in W + 1 still returns the old state; from W + 2 it returns the new one.

Selected sub-cores start on the same clock, two clocks after the write

T2 RUN_MASK same-clock start · one cell = one clk



Strobes in W + 1; both sub-cores execute ENTRY in W + 2; its pin action is on the pads in W + 3

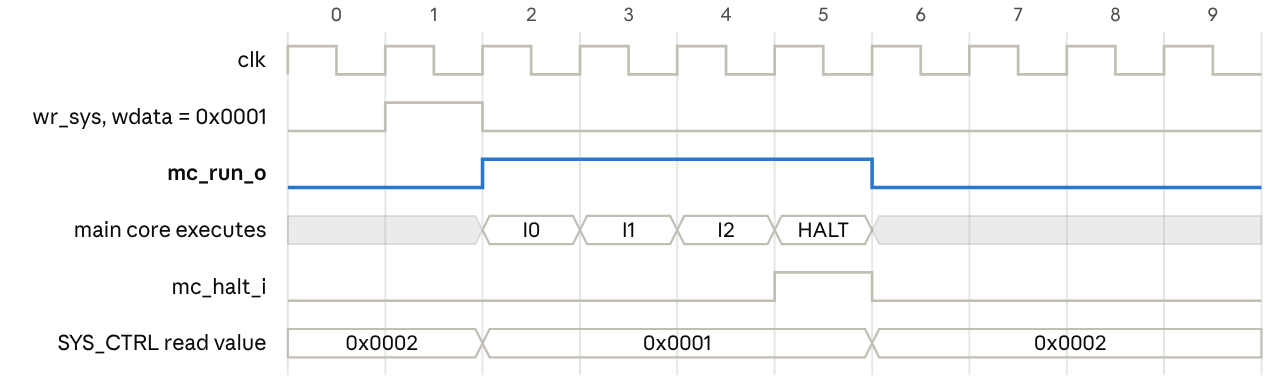
T2 · RUN_MASK = 0x0003 starts SC0 and SC1 together

7.4 Main core: SYS_CTRL[0]/[1] and the HALT instruction

Clock	Event
W	`mc_set` or `mc_clr`
W + 1	`mc_run_o` has its new value; first (or no) instruction executes. SYS_CTRL reads the new state
H (HALT executes)	`mc_halt_i` = 1; PC advances past the HALT
H + 1	`mc_run_o` = 0, as in `mc.py`

The main core runs one clock after SYS_CTRL[0] and stops after its HALT

T3 Main-core run and halt · one cell = one clk



First instruction in W + 1. HALT executes in clock 5; the core is halted from clock 6 with the PC after the HALT

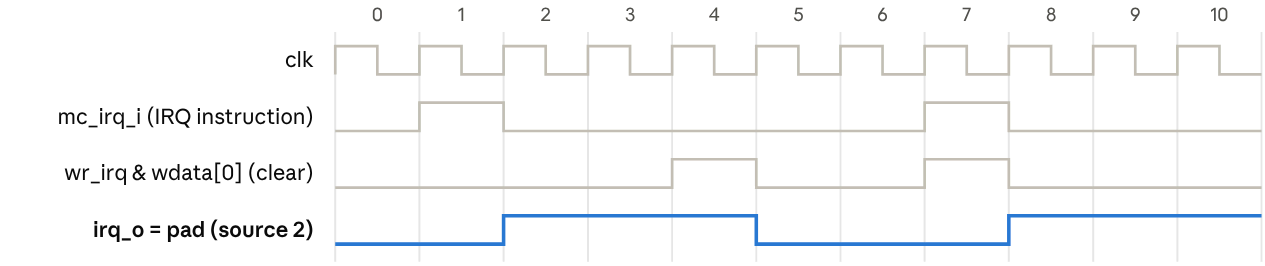
T3 · main core started by SYS_CTRL[0], stopped by HALT

7.5 IRQ

Clock	Event
X (IRQ executes)	`mc_irq_i` = 1
X + 1	`irq_o` = 1; a pad mapped to source 2 is high in this clock (pin layer is combinational, PL-05)
W (clear write)	`wr_irq` & wdata[0]
W + 1	`irq_o` = 0, unless `mc_irq_i` was 1 in W

IRQ stays set until cleared, and a new IRQ wins over a clear

T4 IRQ set, clear and set-wins · one cell = one clk



Clock 1: set. Clock 4: clear. Clock 7: set and clear together, IRQ ends set

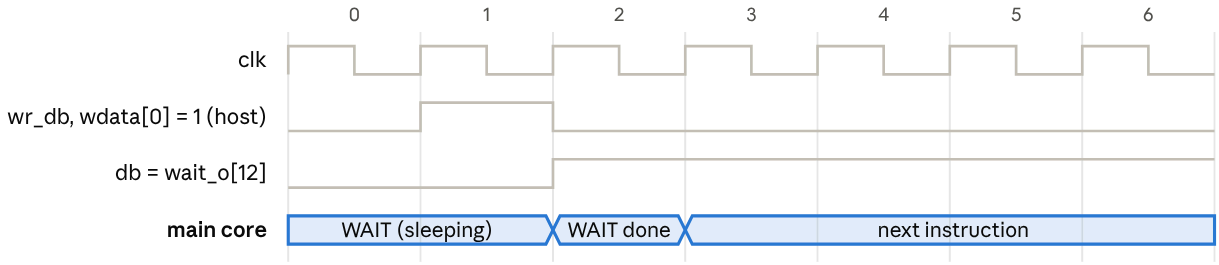
T4 · IRQ set, clear and set-wins

7.6 Doorbell

Written in W, `wait_o[12]` = 1 in W + 1; a main core sleeping in WAIT on bit 12 completes the WAIT in W + 1 and executes the next instruction in W + 2.

A doorbell write wakes a sleeping WAIT in the next clock

T5 Doorbell to WAIT wake-up · one cell = one clk



Written in W (clock 1); WAIT sees it and completes in W + 1; the next instruction executes in W + 2

T5 · doorbell write wakes WAIT

8. Clocking and reset

The block has one clock and resets every flop to 0.

- One clock (`clk`), no generated or gated clocks (G-01). No asynchronous inputs: every input comes from a flop or from logic in the `clk` domain.
- All 30 flops reset to 0 by `rst_n` . After reset: prescaler at 0 and running, no strobes, main core halted, IRQ 0, doorbell 0 (G-02, SYS-01).
- The prescaler is never stopped. Only SYS_CTRL[2] changes its phase.

9. Area (G-11)

The estimate is about 2.7k µm² before CTS, within the 14.0k budget shared with the host link and the bus.

Item	Count	Unit µm²	Area µm²
dfrbpq flops	30	48.9888	1,470
Prescaler: 12-bit incrementer, restart clear, tick OR	≈ 11 XOR2, 22 AND2, 3 OR2	9–15	≈ 390

Item	Count	Unit µm²	Area µm²
Offset decode, write strobes, run/halt strobe gating	≈ 22 gates	7–15	≈ 220
mc_run, irq, db next-state	≈ 8 gates, 1 MUX2	9–18	≈ 100
Read mux (AND-OR, 16 bits) and EVENTS[14]	≈ 30 AND2, 16 OR, 4 INV	5–15	≈ 480
Total before CTS			≈ 2.7k

Flop, MUX2, AND2 and XOR2 areas are from the project's area_model.py. INV, OR and wide-gate areas are assumptions, and the gate counts are hand estimates (±30 %). The tied-off inputs and outputs cost nothing. With the host link at ≈ 3.8k, about 7.5k of the 14.0k budget (SYS-06) is left for the bus.

10. Timing (internal)

No path inside the block is long; the paths to watch cross into the main core and the interconnect.

- Longest register-to-register path inside the block: cnt[0] → c4 → c8 → c12 → OR → tick[2] D (three AND4 levels and an OR), or the incrementer's bit 11, about the same depth.
- `mc_halt_i` / `mc_irq_i` → `mc_run` / `irq` D: 2–3 gate levels here, but they start late (SRAM clock-to-out, then instruction decode in the main core). Budget them in the main-core chapter.
- `wait_o`: sub-core flops → FIFO count compare (sub-core) → wire → here (bit 14: two OR levels) → main core WAIT compare. Purely combinational through this block; it crosses the die, so constrain it as a main-core path.
- `rdata_o`: off_q (flops) → 4-bit decode → AND-OR (3 levels) → interconnect read mux → master's read-data register (host link `sr`). Starts at a flop, so most of the 20 ns is left for the interconnect.
- `req_i`, `dir_i`, `addr_i[3:0]` → write strobes → flop D: decode plus 2 gate levels. Budget the interconnect part in the bus chapter.

- `tick_o`, `sc_run_o`, `sc_halt_o`, `irq_o`, `mc_run_o` come straight from flops. Same-clock arrival at the four sub-cores (SB-03, SB-04) is an ordinary setup constraint on these nets.

11. Software rules

Six rules follow from the decisions; firmware and host software must keep them.

1. R-1 Never set the run and the halt bit of one target in one RUN_MASK or SYS_CTRL write. If it happens, the target is halted (the opposite of CTRL, where RUN wins).
2. R-2 EVENTS[14] is 1 while any sub-core is halted, including unused sub-cores and a 32/0 partner. Programs that halt on an error should `MARK` or `NTF` first; firmware waits on the FIFO B or notify bit and then reads STATUS.
3. R-3 Doorbell convention: the host writes 1, the main core writes 0 before it handles the request (so a second ring during handling is not lost).
4. R-4 IRQ is a level and stays set until a write of 1 clears it. If a new IRQ arrives in the clock of the clear, IRQ stays set.
5. R-5 A RUN_MASK read in the clock right after a RUN_MASK write returns the old state (only the main core can read that fast).
6. R-6 Use `psc` ≠ 0 only where a phase error of up to one tick period is acceptable, or restart the prescaler with SYS_CTRL[2] first.

12. Verification plan (VER-04, VER-05, VER-06)

Eleven directed tests cover the port, every register, the prescaler, both run controls and the WAIT vector, then repeat on the netlist.

Test	Check
T0 Bus port	wready_o = rready_o = 1 in every clock; a write takes effect at the end of its req clock; rdata valid exactly one clock after the read's req clock; addr[15:4] and wdata[15:8] have no effect; reads have no side effect
T1 Reset	ID reads 0x5010, SYS_CTRL 0x0002, RUN_MASK 0x00F0, IRQ 0, DOORBELL 0; no strobes; offsets 0x6–0xF read 0 and ignore writes

Test	Check
T2 Prescaler	Free-running: each tick period exactly 16 / 256 / 4096 clocks over 3 × 4096 clocks. Restart at random phases: all ticks in W + 1, periods exact afterwards
T3 RUN_MASK	Every run/halt bit pattern: strobes for one clock in W + 1; run + halt for one n gives halt only; a SYS_CTRL write raises no sub-core strobe
T4 Same-clock start	With real sub-cores: all selected sub-cores execute their entry state in W + 2; entry actions on pads in W + 3
T5 Main core	SYS_CTRL run, halt, run + halt (halted); HALT instruction; HALT instruction in the clock of a run write (halted); reads follow the state
T6 IRQ	Set by the instruction; cleared by a write of 1 from host and from main core; a write of 0 does nothing; set + clear in one clock leaves it set; pad source 2 follows
T7 Doorbell	Write 1 / 0 from either master; WAIT bit 12; wake-up clock of figure T5
T8 EVENTS	Each of the 15 bits against its source; bit 15 = 0; bit 14 with halted only, error only, both
T9 S-3	Run strobe to a 32/0 partner is ignored (sc_pair test)
T10 Gate level	Same tests on the netlist, no X after reset

Model comparison (S-5): the RTL prescaler runs from reset and the reference model's starts at 0, so the harness sets the model's prescaler count `g` to the RTL `cnt` in the clock the sub-cores start. Test side only.

Reference: `sc_gates.py` implements §4–§5 and passes its checks: tick alignment with the `pe.py` definition (g mod 16 / 256 / 4096 = 0) over 3 × 4096 clocks, 200 random restarts, strobe width and halt-wins, main-core run/halt including the HALT-instruction case, IRQ

set-wins, doorbell, EVENTS, read-back values, read data one clock after the read, tied-off address and data bits, and no read side effects. The script is in the session scratch space; move it into the repository with the RTL.

13. Decisions and open items

Eighteen owner decisions are settled; four decisions taken while writing the spec await the owner, and six items are open outside this block.

Owner decisions, 10 Nov 2026:

ID	Decision	Reason
S-1	EVENTS[14] as written	No area; software rule R-2
S-2	Halt wins in RUN_MASK and SYS_CTRL	Owner: a conflicting write leaves the target stopped
S-3	32/0 partner ignores run	Matches the reference model, which keeps the partner halted
S-4a	DOORBELL plain read/write bit	Simplest; no master ID
S-4b (A-CHG-3)	Any master clears IRQ	§1.1
S-4c	IRQ set wins over clear	No lost interrupt
M-1 (A-CHG-4)	SYS_CTRL and RUN_MASK read live state, running and halted bits	§1.1
M-2	ID = 0x5010	'P', version 1.0
D-1	Ticks registered	Flop-to-flop paths across the die
D-2	Restart: ticks in the next clock	Matches the model's start at 0
D-3	Sub-core run/halt strobes registered	Same-clock start becomes a plain setup constraint; one extra clock of start latency
D-4	WAIT vector combinational	Cycle match with `mc.py`
D-5	Main core's halted state lives in sys_ctrl	Owner decision

ID	Decision	Reason
D-6	0x1C00–0x1C0F never wait	BUS-06
CRC	CRC unit under main_core	Matches its function and its area budget
S-5	Harness seeds the model's prescaler	No hardware change
dir	Master-side `dir` 1 = write	As in the host-link spec
Bus port	Same 7-signal interface as the host link; unused signals tied off inside (§2.1)	Owner decision

Decisions taken while writing; none is visible in the architecture text. Pick Confirmed or Rejected in the last column:

ID	Decision	Reason	Owner review
D-SYS-7	SYS_CTRL[0]/[1] act on `mc_run` directly, without strobe flops. D-3 then covers the 8 sub-core strobes only	With D-5 the flop is local, so strobe flops would add 2 flops and one clock for nothing (area first). The main core starts in W + 1, sub-cores in W + 2	Awaiting owner
D-SYS-8	A HALT instruction in the clock of a SYS_CTRL run write: the core ends halted	Extends S-2 (halt wins) to the instruction	Awaiting owner
D-SYS-9	`mc_run` is stored with run polarity and all flops reset to 0	One flop type (dfrbpq), as in the host link	Awaiting owner
D-SYS-10	Read data timing: register the 4-bit offset (`off_q`) and drive `rdata_o` from it in the clock after acceptance, instead of registering 15 bits of read data	Meets "rdata valid the clock after rready" with 4 flops instead of 15 (area first); no read side effects, so sampling one clock later is invisible (§5.6)	Awaiting owner

Open and unverified:

- AS-1: how the interconnect drives the port (one-hot `req`, rdata select in the clock after the read), §2.1, until the bus chapter exists.
- The split of "halted" and "error" into two wires per sub-core (§2.2) changes the status interface of SC-STS-05; to be taken into the sub-core chapter.
- Main-core behaviour when `mc_run_o` falls in the middle of a multi-clock instruction (main-core chapter).
- Priority inside a sub-core between a RUN_MASK strobe and a CTRL write, HALT micro-op or breakpoint in the same clock (sub-core chapter). The main core can cause this by writing RUN_MASK and CTRL in consecutive clocks.
- INV, OR and wide-gate cell areas; gate counts are hand estimates.
- Documents to amend for A-CHG-3, A-CHG-4 and the CRC move: architecture-1.0.md §6.4, uarch-requirements-1.0.md §2 and §6, claude/uarch-memory-map.md §5 and §7.

Appendix A. WaveDrom sources

Each block renders with WaveDrom 3.x (wavedrom.com/editor.html or any WaveDrom plugin); one cell = one clk period. Figures T1–T6 in §7 are drawn from these sources.

T1 Prescaler restart (wd1_prescaler)

```
{
 "signal": [
  {"name": "clk", "wave": "p........"},
  {"name": "wr_sys & wdata[2]", "wave": "010......"},
  {"name": "cnt", "wave": "=========", "data": ["n", "n+1", "0", "1", "2",
"3", "4", "5", "6"]},
  {"name": "tick[0] (/16)", "wave": "0.10....."},
  {"name": "tick[1] (/256)", "wave": "0.10....."},
  {"name": "tick[2] (/4096)", "wave": "0.10....."}
 ],
 "head": {"text": "Prescaler restart by SYS_CTRL[2] (write accepted in clock
1 = W)", "tick": 0},
 "foot": {"text": "All three ticks pulse in W + 1 (cnt = 0); the next /16
tick is in W + 17, /256 in W + 257, /4096 in W + 4097"}
}
```

T2 RUN_MASK same-clock start (wd2_run_mask)

```
{
 "signal": [
  {"name": "clk", "wave": "p......."},
  {"name": "wr_rm, wdata = 0x0003", "wave": "010....."},
  {"name": "sc_run_o[0]", "wave": "0.10...."},
  {"name": "sc_run_o[1]", "wave": "0.10...."},
  {"name": "SC0 halted", "wave": "1..0...."},
  {"name": "SC1 halted", "wave": "1..0...."},
  {"name": "SC0 state", "wave": "=...==..", "data": ["ENTRY (held, no
actions)", "S1", "S2"]},
  {"name": "SC1 state", "wave": "=...==..", "data": ["ENTRY (held, no
actions)", "S1", "S2"]},
  {"name": "pads (entry action)", "wave": "=...=...", "data": ["idle",
"action of ENTRY"]}
 ],
 "head": {"text": "RUN_MASK = 0x0003: SC0 and SC1 start on the same clock
(write accepted in clock 1 = W)", "tick": 0},
 "foot": {"text": "Strobes in W + 1; both sub-cores execute ENTRY in W + 2;
its pin action is on the pads in W + 3"}
}
```

T3 Main-core run and halt (wd3_mc_run)

```
{
 "signal": [
  {"name": "clk", "wave": "p........."},
  {"name": "wr_sys, wdata = 0x0001", "wave": "010......."},
  {"name": "mc_run_o", "wave": "0.1...0..."},
  {"name": "main core executes", "wave": "x.====x...", "data": ["I0", "I1",
"I2", "HALT"]},
  {"name": "mc_halt_i", "wave": "0....10..."},
  {"name": "SYS_CTRL read value", "wave": "=.=...=...", "data": ["0x0002",
"0x0001", "0x0002"]}
 ],
 "head": {"text": "Main core started by SYS_CTRL[0], stopped by its own HALT
instruction", "tick": 0},
 "foot": {"text": "First instruction in W + 1. HALT executes in clock 5; the
core is halted from clock 6 with the PC after the HALT"}
}
```

T4 IRQ set, clear and set-wins (wd4_irq)

```
{
 "signal": [
  {"name": "clk", "wave": "p........."},
  {"name": "mc_irq_i (IRQ instruction)", "wave": "010....10.."},
  {"name": "wr_irq & wdata[0] (clear)", "wave": "0...10.10.."},
  {"name": "irq_o = pad (source 2)", "wave": "0.1..0..1.."}
 ],
 "head": {"text": "IRQ: set by the instruction, cleared by a write of 1, set wins in the same clock", "tick": 0},
 "foot": {"text": "Clock 1: set. Clock 4: clear. Clock 7: set and clear together, IRQ ends set"}
}
```

T5 Doorbell to WAIT wake-up (wd5_doorbell)

```
{
 "signal": [
  {"name": "clk", "wave": "p......"},
  {"name": "wr_db, wdata[0] = 1 (host)", "wave": "010...."},
  {"name": "db = wait_o[12]", "wave": "0.1...."},
  {"name": "main core", "wave": "=.==...", "data": ["WAIT (sleeping)", "WAIT done", "next instruction"]}
 ],
 "head": {"text": "Doorbell wakes a main core sleeping in WAIT with bit 12 in its mask", "tick": 0},
 "foot": {"text": "Written in W (clock 1); WAIT sees it and completes in W + 1; the next instruction executes in W + 2"}
}
```

T6 Bus write and read (wd6_bus)

```
{
 "signal": [
  {"name": "clk", "wave": "p......"},
  {"name": "req_i", "wave": "01010.."},
  {"name": "dir_i", "wave": "x1x0x.."},
  {"name": "addr_i", "wave": "x=x=x..", "data": ["0x1C04", "0x1C05"]},
  {"name": "wdata_i", "wave": "x=x....", "data": ["0x0001"]},
  {"name": "wready_o, rready_o", "wave": "1......"},
  {"name": "db", "wave": "0.1...."},
  {"name": "off_q", "wave": "x.=x=x.", "data": ["4", "5"]},
  {"name": "rdata_o (used)", "wave": "x...=x.", "data": ["EVENTS"]}
 ],
 "head": {"text": "Bus write (DOORBELL = 1) then read (EVENTS), no wait",
"tick": 0},
 "foot": {"text": "Write accepted in clock 1, db = 1 from clock 2. Read
accepted in clock 3, off_q = 5 and rdata valid in clock 4"}
}
```